CFM Agent v6.0

Developer Implementation Specification: the complete v5.2-to-v6.0 delta, architecture, component contracts, numerical transformations, configuration file, design-decision register, and the locked development plan

FINAL NORMATIVE SPECIFICATION, LOCKED FOR IMPLEMENTATION (2026-09-16)

This document is the single authority for CFM Agent v6.0. It replaces the Locked Specification Archive, the Full Technical Specification, the Developer Implementation Specification (Revisions 1 and 2), the Specification Summary, and the Readiness Assessment of 2026-09-15; everything accepted from those documents is folded in here, and where any of them differs from this document, this document governs. Design choices and the reasons for them are recorded in Section 12 and mirrored in the package as DECISIONS.md; every tunable value is held in the package's settings.toml, reproduced in Section 7. The implementation ships this document as SPECIFICATION.pdf. It describes locked requirements; it does not claim that v6.0 has been implemented or validated.

0. How to read this document

- Section 1 specifies the numerical foundation; Sections 2 to 4 the diagnostics, transformations, research, evidence, LLM roles, and E2B controls; Section 5 the end-to-end flow; Section 6 ownership and layout; Section 7 the configuration file; Section 8 the audit record; Sections 9 to 11 the development plan, gates, and exclusions; Section 12 the design-decision register; Section 13 the glossary.
- Values printed in tables of Sections 1 to 4 are the locked defaults. The normative home of every value is settings.toml (Section 7); a table and the file must never disagree, and a test enforces that the loaded defaults equal the file.
- Percentiles, bands, and multipliers quoted with a date are illustrative measurements on the history to that date. The engine recomputes them at every cutoff.
- Decision identifiers DD-nn refer to Section 12.

1. Changes and specifications for v6.0 versus v5.2

1.1 Package and scope boundary

Area	v5.2	Locked v6.0
Package	cfm_agent_v_5_2	New isolated sibling cfm_agent_v_6_0 under implementations/ energy_oil_forecasting; shared main, v5.0, v5.2, and cfm_coach protected byte-for-byte (DD-01)
Numerical models	AutoARIMA, Kalman, price-level LightGBM (shared Darts library)	Historical Bootstrap Return, Sparse AR Return, Lightweight LightGBM Return, all package-owned (NumPy and LightGBM only) (DD-03)
Weights	One third each	0.50 / 0.25 / 0.25 (DD-04)
Simulation	Model-specific price-level samples	Exactly 1,000 daily return paths per component, 21 steps long
Horizons	5, 10, 21 business days	Retained
Diagnostics	Nine fields	Multi-window returns, volatility, volatility z-scores, drawdowns, jump z-scores, current state, data quality, and the historical return and deviation distributions
Transformations	Width-fraction centre steps; hand-set interval multipliers 0.7 to 1.3	Historical-percentile centre steps; coverage-shaped uncertainty ladder derived from history (DD-09, DD-10)
Evidence	v5.2 tiers; strict Tier 3	Tier 3 simplified to two publishers and 0.80 confidence; passage linkage added (DD-12, DD-13)
LLM architecture	LITE_MODEL alias for two roles; three hard-coded strings; main call temperature unset, no timeout	Five independently configurable roles with parameter forwarding to the invocation layer (DD-14)
Configuration	Python defaults inside config.py; literals in several modules	Every tunable value in settings.toml; Python holds structural constants only (DD-16)
E2B	LLM-authored code, no caps, internet on, count-only audit	Python-enforced budgets, static prohibited-operation screen, Python-defined payload, full audit (DD-15)

Audit	22 metadata keys; settings not serialised by the package	Expanded numerical, diagnostic, evidence, LLM, E2B, transformation, and output audit; settings file hash, resolved values, and overrides recorded

1.2 Common numerical suite contract

Requirement	Locked value / behaviour
Components	Historical Bootstrap Return; Sparse AR Return; Lightweight LightGBM Return
Configured weights	0.50, 0.25, 0.25 (keys bootstrap, sparse_ar, lightgbm)
Path count	Exactly 1,000 return paths per component
Path length	21 steps; horizons 5, 10, and 21 are read off the same paths at columns 5, 10, and 21
Step and business-day convention	A step is one WTI observation (a trading day in the shared yfinance series). Horizon h means h steps, and historical h-step returns use h observations. Forecast dates are labelled cutoff plus h pandas business days (Monday to Friday, no holiday calendar), the convention of the shared library and its cutoff enforcer. A week containing an exchange holiday therefore labels a forecast one calendar day late; accepted and recorded (DD-19)
Return definition	Daily simple return P(t) / P(t-1) - 1 on floored prices
Price reconstruction	Latest cutoff-safe WTI multiplied by the cumulative product of (1 + simulated return)
History window	Setting history_window_business_days, default 0 = the entire cutoff-safe history; applies to bootstrap sampling, model training, and transformation tables alike (DD-07)
Quantiles	Eleven levels 0.05, 0.10, 0.20, 0.30, 0.40, 0.50, 0.60, 0.70, 0.80, 0.90, 0.95 for every component and the ensemble (DD-05); computed with numpy.quantile, method "linear", on the 1,000 simulated prices at the horizon column
Point forecast	P50, read after the ordering guard (Section 1.6)
Input / simulated / final floors	Configurable; default $1.00 per barrel each (DD-06). Input floor applied before returns, diagnostics, and fitting; simulated floor applied to every simulated price before quantiles; final floor applied after transformation. The settings loader rejects a final floor above the simulated floor, which guarantees Section 3.5
Return clipping	Prohibited. Returns are never floored, clipped, capped, winsorised, or demeaned (DD-08)
RNG	Deterministic task-derived suite seed; one component generator per model spawned from it (Section 1.7)
Component failure	Exclude the failed component, record a failure code and the reason, renormalise the successful positive weights
Complete failure	Fail closed if every positively weighted component fails
Strict mode	Setting require_all_components, default false; when true any component failure fails the suite
Execution count	The authoritative numerical suite executes exactly once per workflow; pre-execution attempts capped at 2
LLM snapshot	The tool result shown to the LLM carries at most 520 observations per series (max_data_rows_per_series); models always train on the history window above. Paths never leave Python

1.3 Historical Bootstrap Return

Parameter	Specification
Input	Cutoff-safe daily WTI prices, floored at the input floor, then daily simple returns
Block construction	All eligible overlapping five-step return blocks (start indices 0 to n-5)
Sampling	Uniform with replacement using the component generator; five blocks per path, concatenated and truncated to 21 returns
Drift	Sampled returns are used as observed. The history's mean daily return and the compounding drag are part of the forecast
Price path	Latest WTI multiplied by the cumulative product of (1 + return); simulated floor applied to every price
Failure rule	Fewer than block_length_business_days + min_training_rows_margin returns (55 by default): failure code insufficient_history
Audit	Eligible observations, eligible block count, block size, path count, component seed, floor settings and activation counts, component quantiles

1.4 Sparse AR Return

Parameter	Specification
Target	Next daily WTI simple return
Features	The single lagged daily returns r(t-1), r(t-5), r(t-10), r(t-21), r(t-63), r(t-126), r(t-252) (setting return_lags)
Estimator	Closed-form ridge regression; penalty ar_ridge_penalty (1e-5) on the seven coefficients; unpenalised intercept
Innovations	Overlapping five-step blocks of in-sample residuals, sampled with the component generator
Forecast mode	Recursive: at each future step the lag vector is built from observed returns and returns already simulated on that path; the conditional ridge return plus the sampled residual is appended and the recursion continues
Failure rule	Fewer than max(return_lags) + block_length_business_days + min_training_rows_margin returns (307 by default): failure code insufficient_history
Output	Compound each path into prices, apply the simulated floor, read the eleven quantiles at horizons 5, 10, 21

1.5 Lightweight LightGBM Return

Parameter	Specification
Target	Daily WTI simple return; L2 (regression) objective; a single predicted value per step, never quantile objectives
Features	The seven WTI lagged returns of Section 1.4 plus the seven covariate values as supplied by the shared data layer (one feature each, fourteen in total, covariate order as listed in settings). The data layer already lags every covariate by one business day
Hyperparameters	250 estimators; learning rate 0.03; 7 leaves; maximum depth 3; deterministic=True; force_row_wise=True; num_threads=1; random_state = component seed; verbose=-1
Residual blocks	Overlapping five-step blocks of in-sample residuals
Paths	Exactly 1,000; recursive, with simulated WTI returns feeding future WTI lag features
Missing-data policy	A covariate absent from the data service makes the component invalid (failure code covariate_missing). A covariate present with gaps is forward-filled up to covariate_forward_fill_limit (5) consecutive observations; training rows still missing any feature are dropped
Eligibility after alignment	The aligned training frame that remains after forward-fill and row dropping must hold at least max(return_lags) + block_length_business_days + min_training_rows_margin rows (307 by default); otherwise failure code insufficient_aligned_rows. The audit records rows before alignment, filled cells per covariate, rows dropped, and rows remaining

Covariate	Series (shared data layer)	Historical input	Future simulation assumption
Brent return	brent_log_ret_1b_l1b	Cutoff-safe	0
Natural-gas return	natgas_log_ret_1b_l1b	Cutoff-safe	0
Gasoline return	gasoline_log_ret_1b_l1b	Cutoff-safe	0
Gold return	gold_log_ret_1b_l1b	Cutoff-safe	0
Dollar-index return	dollar_index_log_ret_1b_l1b	Cutoff-safe	0
Oil-curve contango	oil_curve_contango_l1b (log USL/USO)	Cutoff-safe	Hold latest cutoff-safe value
VIX	vix_level_l1b	Cutoff-safe	Hold latest cutoff-safe value

- Align all covariates to the cutoff-safe WTI observation dates deterministically (inner join on the WTI dates); never use post-cutoff covariate observations.
- The shared data layer (implementations/energy_oil_forecasting/data.py) is used unchanged; no new data source is added.

1.6 Component validation, failure handling, and ensemble

Validation	Requirement
Paths	Shape exactly (1000, 21)
Numerical values	Finite return paths and finite reconstructed prices (failure code non_finite_output)
Price constraints	Every simulated price at or above the simulated floor (failure code floor_violation)
Horizon outputs	5, 10, 21 steps, dated cutoff plus h business days
Quantiles	All eleven levels present, finite, non-decreasing in the level (failure code quantile_crossing)

Point forecast	Exactly equals the component P50
Metadata	Component identity, component seed, forecast-affecting settings, path count, training audit
Failure disposition	A component that raises or fails any check is excluded before the ensemble; its failure code enters the suite identity, its verbatim reason enters the audit only

Order of operations. The sequence below is normative; nothing may be reordered or repeated.

```
1  simulate     returns[1000, 21] per component, from the component generator
2  compound     prices = last_price * cumprod(1 + returns); prices = max(prices, simulated_price_floor)
3  quantiles    Q_i(q, h) = numpy.quantile(prices[:, h-1], q, method="linear"), eleven levels
4  validate     shape, finite, floor, non-decreasing; a failure becomes a disposition with a failure code
5  point        Point_i(h) = Q_i(0.50, h)
6  pool         Q_e(q, h) = sum over active i of w_i(active) * Q_i(q, h);  w_i(active) = w_i / sum(active w)
7  guard        cumulative maximum over q. Defensive only: a weighted sum of non-decreasing vectors is
                non-decreasing under IEEE arithmetic, so the guard cannot change any value. Any activation
                is recorded as ensemble_guard_applied = true and is a build failure
8  point        Point_e(h) = Q_e(0.50, h), read after the guard; this is the baseline P50 of Section 3
9  transform    Section 3: the P50 is moved first, then the quantile distances are scaled around it
10 final floor  Q_f(q, h) = max(Q_f(q, h), final_price_floor); ordering re-checked; Point_f(h) = Q_f(0.50, h)
```

Failure code	Raised by	Meaning
insufficient_history	Bootstrap, Sparse AR	Fewer WTI returns than the component's minimum
insufficient_aligned_rows	LightGBM	Aligned training frame below the minimum after forward-fill and drops
covariate_missing	LightGBM	A configured covariate is absent from the data service
non_finite_output	Any	A non-finite return, price, or quantile
floor_violation	Any	A simulated price below the simulated floor after flooring (a bug)
shape_mismatch	Any	Paths not exactly (1000, 21)
quantile_crossing	Any	Component quantiles decrease in the level
library_error	Any	Any other exception; the exception class name is appended, the message is audit-only

- Ensemble construction uses already executed component results and never reruns a model.
- If Historical Bootstrap fails while both 25% models succeed, each active weight becomes 0.50.
- If all positively weighted models fail, or require_all_components is true and any model fails, the suite fails closed.

1.7 Deterministic RNG, canonical serialization, and model-suite identity

Identity input	Required binding
Task	Target series, cutoff / as-of, frequency, horizons
Settings	Every field tagged [identity] in settings.toml (Section 7), enumerated in config.IDENTITY_FIELDS
Data	Covariate identities and the data fingerprint below
Results	Per component: status, failure code, component seed, and the eleven quantiles per horizon; active weights; dispositions

Canonical serialization. One encoder, fingerprints.canonical_json, produces every identity payload (DD-17):

```
canonical_json(payload) = json.dumps(payload, sort_keys=True, separators=(",", ":"),
                                     ensure_ascii=True, allow_nan=False, default=encode)
encode:  date -> "YYYY-MM-DD";  timestamp -> "YYYY-MM-DDTHH:MM:SS"
         (tz-aware values converted to UTC, then made naive)
         numpy integer / floating -> int / float;  numpy array, tuple -> list;  set -> sorted list
         any other type -> TypeError (never str())
floats:  Python repr (shortest round trip); -0.0 is written as 0.0; NaN and infinities are rejected
missing: null, from a fixed field set; keys are never omitted
strings: NFC-normalised; free-form error text is never part of an identity payload
identity = "prefix:sha256:" + sha256(canonical_json(payload))
suite_seed = int(sha256 hex[0:8], 16)                      first 32 bits
component generators = numpy.random.SeedSequence(suite_seed).spawn(3)
                       in the order bootstrap, sparse_ar, lightgbm
```

- Data fingerprint payload: {"cutoff", "window", "series": {series_id: [["YYYY-MM-DD", value or null], ...]}} for the WTI target (floored at the input floor, windowed) and the seven covariates as served by the data layer, before forward-fill and row dropping. Fill and drop are deterministic functions of this payload and the identity settings.
- Identical task inputs, identity settings, and data fingerprint reproduce component seeds, paths, quantiles, failure

dispositions, active weights, ensemble quantiles, diagnostics, and the model-suite ID, in-process and across fresh processes. Changing any identity setting changes the model-suite ID; LLM, E2B, research, audit, and validation settings do not. Wall-clock timestamps may differ but never determine numerical output.
- Each component's paths depend only on its own generator, so they are reproducible regardless of execution order or of another component's failure.

1.8 Recorded numerical properties under the locked choices

These are consequences of Sections 1.2 to 1.5 measured on the 2026-09-14 history. They are documented in CALIBRATION.md and are not defects to be corrected in this release.

- **April 2020.** CL=F settled at -$37.63 on 2020-04-20. Floored to $1.00, the daily returns are -94.5% on 2020-04-20 and +901.0% on 2020-04-21. Both remain in every full-history fit. Measured effect on the Sparse AR fit: residual standard deviation 12.5% per day (2.6% with those two returns excluded); intercept +0.224% per day (+0.055% excluded). In the bootstrap, 2 of 1,000 21-step paths finished above twice the last price (maximum $1,325 from $102.58); P90 is unaffected.
- **Sensitivity report.** release.py sensitivity reruns the suite and the transformation tables at the release cutoff with the returns on sensitivity_excluded_dates removed and writes the comparison (component and ensemble P10/P50/P90 at 5/10/21, AR intercept and residual standard deviation, centre percentiles, ladder multipliers) to CALIBRATION.md. It is informational and never alters the production model (DD-06).
- **Drift.** The full-history mean daily return is +0.055% per day. The bootstrap median at h=21 sits about +1.25% above the last price. With the AR intercept above at 25% weight, the ensemble P50 is expected to sit roughly $1.50 to $2.00 above the last price at h=21 against a historical mean forward move of +$0.18. v5.2 sat about $2.60 below.
- **Centre asymmetry.** Raw percentiles straddle a positive median, so the down and up steps of the same size differ in magnitude (Section 3.1).

2. Expanded deterministic diagnostics

Diagnostic	Windows (settings) / content	Definition
Cumulative return and trend	diagnostic_return_windows: 1, 5, 10, 21, 63, 126, 252	v(last) / v(last-N) - 1 on floored prices
Annualised realised volatility	diagnostic_volatility_windows: 5, 10, 21, 63, 126, 252	Standard deviation (ddof 0) of the last N daily simple returns, times the square root of trading_days_per_year (252)
Same-horizon volatility z-score	diagnostic_volatility_windows	(current realised volatility over N minus the mean of every historical rolling N-observation realised volatility up to the cutoff) divided by their standard deviation
Drawdown	diagnostic_volatility_windows	v(last) / max(last N values) - 1
Latest daily-return jump z-score	diagnostic_jump_windows: 10, 21, 63, 126, 252	(last daily return minus the mean of the last N daily returns) divided by their standard deviation
Current state	-	Latest WTI price and date, calendar gap in days, the seven current covariate values, curve state (contango, backwardation, or flat when the absolute log ratio is below curve_flat_threshold), VIX level
Historical transformation inputs	5, 10, 21	The HistoricalReturnProfile per horizon (Section 3.4): eligible count, window bounds, median, return percentiles, absolute-deviation bands
Model disagreement	5, 10, 21	Standard deviation of component point forecasts
Data quality	-	Observation counts per series, first and last dates, cutoff, input-floor activation count and dates, covariate fill and drop counts, rows remaining for LightGBM

- Diagnostics are cutoff-safe, deterministic, Python-owned, and use no random numbers. A window longer than the available history yields null, never an error.
- The LLM may interpret diagnostics but may not replace authoritative calculations.
- Diagnostics alone cannot satisfy an evidence tier.

3. Historical forecast transformations

3.1 Centre-action decisions

Centre action	Historical horizon-matched return percentile	h=5 value	h=10 value	h=21 value
Large down	P10	-6.09%	-8.36%	-11.56%
Moderate down	P25	-2.75%	-3.76%	-5.53%
Small down	P40	-0.74%	-0.90%	-1.40%
No change	0% adjustment	0	0	0
Small up	P60	+1.43%	+2.19%	+3.58%
Moderate up	P75	+3.25%	+4.62%	+7.37%
Large up	P90	+6.19%	+8.65%	+12.98%

Values are the full-history percentiles as of 2026-09-14 and are illustrative; the engine recomputes them at every cutoff. The percentile per action is the setting center_percentiles.

```
HistoricalReturn(h)        = percentile(eligible overlapping h-step WTI returns, selected percentile)
GuardedReturn(h)           = HistoricalReturn(h) if sign(HistoricalReturn(h)) == direction(action)
                             else 0                                          (directional guard)
RawCenterAdjustment(h)     = BaselineP50(h) * GuardedReturn(h)
NoveltyAdjusted(h)         = RawCenterAdjustment(h) * NoveltyMultiplier
EmergencyCap               = min(emergency_center_cap_usd, emergency_center_cap_price_fraction * latest WTI)
CappedCenterAdjustment(h)  = clip(NoveltyAdjusted(h), -EmergencyCap, +EmergencyCap)
AdjustedP50(h)             = BaselineP50(h) + CappedCenterAdjustment(h)
```

- Use all eligible, overlapping, cutoff-safe, horizon-matched cumulative WTI returns from the history window.
- The LLM proposes only the categorical action. Python selects the percentile and performs all arithmetic.
- Directional guard: a down action can never move the centre up and an up action can never move it down. When the selected percentile has the wrong sign the adjustment is exactly zero and a warning is recorded (DD-09).
- For no_change the centre adjustment is exactly zero.
- The adjusted P50 becomes the centre for the uncertainty transformation.

3.2 Novelty multipliers

Novelty classification	Multiplier (setting novelty_multipliers)
likely_new_relative_to_model_data	1.00
possibly_partly_reflected	0.75
likely_reflected_in_model_data	0.50
indeterminate	0.50

- Apply novelty after selecting and converting the historical centre percentile and before the emergency cap.
- All four classifications may accompany a non-neutral action. The v5.2 rule that only likely-new or possibly-partly-reflected novelty may act is removed (DD-11).
- Novelty is validated as an enumeration and recorded; it never qualifies evidence, never raises the evidence tier, and is not capped by tier.

3.3 Uncertainty-action decisions: the coverage-shaped ladder

Each uncertainty action names a nominal historical coverage: the two-sided band that contained that share of all eligible horizon-matched WTI moves around their median. The ensemble's P10-to-P90 band is by construction an 80% band, so the unchanged action is anchored at the historical 80% band. The multiplier for a rung is the width of its band relative to the anchor band, and it scales the ensemble's own quantile distances around the adjusted P50 (DD-10).

```
HistoricalMedian(h)      = median(eligible overlapping h-step returns)
AbsoluteDeviation(j, h)  = abs(HistoricalReturn(j, h) - HistoricalMedian(h))
Band(h, c)               = percentile(AbsoluteDeviations(h), c)          half-width of the c% band
Multiplier(h, action)    = Band(h, Coverage[action]) / Band(h, AnchorCoverage)
AdjustedQuantile(q, h)   = AdjustedP50(h)
                           + Multiplier(h, action) * [ BaselineQuantile(q, h) - BaselineP50(h) ]

Coverage (setting uncertainty_coverage) = { substantially_narrower: 65, moderately_narrower: 70,
    small_narrower: 75, unchanged: 80, small_wider: 85, moderately_wider: 90, substantially_wider: 95 }
AnchorCoverage (setting uncertainty_anchor_coverage) = 80
```

Uncertainty action	Nominal coverage	h=5	h=10	h=21	v5.2 constant
Substantially narrower	65%	0.69	0.71	0.73	0.70
Moderately narrower	70%	0.77	0.79	0.81	0.80

Small narrower	75%	0.87	0.89	0.90	0.90
Unchanged	80%	1.00	1.00	1.00	1.00
Small wider	85%	1.14	1.15	1.14	1.10
Moderately wider	90%	1.36	1.36	1.35	1.20
Substantially wider	95%	1.74	1.73	1.69	1.30

Multipliers are the full-history values as of 2026-09-14 and are illustrative; the engine recomputes them at every cutoff. WTI's fat tails make widening rungs larger than v5.2's constants while narrowing rungs are close to them.

- The seven coverages and the anchor are validated settings; the ladder must be strictly increasing and the anchor must sit between the narrowing and widening rungs.
- A wider action always yields a multiplier of at least 1.0 and a narrower action at most 1.0. This holds by construction; the engine asserts it and clamps only on exact ties, recording a warning.
- Unchanged yields exactly 1.0 and preserves the baseline quantile distances around the adjusted P50.
- Narrowing requires evidence resolving a specific uncertainty (the narrowing_uncertainty_resolved field must be non-empty). General confidence is insufficient.
- After transformation, apply the final floor and require finite, ordered quantiles with P50 as the point forecast (Section 1.6, step 10).
- Reason for replacing the earlier dollar-target design: measured on the v6.0 bootstrap at h=21, the P60 and P75 absolute-deviation targets ($8.31 and $11.43) were below the baseline half-width ($13.84), giving scale factors 0.60 and 0.83 that the wider-cannot-narrow rule turned into no-ops, while P40, P25, and P10 gave 0.38, 0.24, and 0.10. The baseline half-width sits at the 80th to 84th percentile of absolute deviation, not the 50th.

3.4 Historical return profile, minimum history, and neutral forcing

- For each horizon Python computes a HistoricalReturnProfile: eligible count, window start and end, median, the six centre percentiles, and the absolute-deviation bands at the seven coverages and the anchor. The profile is a diagnostic, an input to the engine, and part of every transformation record, so a recorded forecast can be replayed exactly from the record alone.
- Fewer than min_historical_returns (500) eligible historical horizon returns, invalid historical values, or a zero or invalid baseline width forces a neutral transformation of both the centre and the uncertainty, with the reason recorded.

3.5 Exact-neutral behaviour

- If center_action = no_change and uncertainty_action = unchanged, reproduce the authoritative ensemble exactly by copying it; do not recalculate, round, interpolate, reconstruct, or numerically approximate the neutral result.
- Exact neutral takes precedence over the final floor, and the two cannot conflict: every baseline quantile is a weighted average of prices already floored at the simulated floor, and the loader requires final_price_floor to be at or below simulated_price_floor, so the final floor can never move a neutral copy (DD-18). The engine still re-asserts equality after the floor and raises if it does not hold, because a violation is a bug, not a forecast.
- Preserve point forecasts, quantiles, dates, active weights, component disposition, and the model-suite ID.

3.6 Regime conditioning (present, off)

Setting historical_conditioning accepts "none" (default) or "volatility_quintile". When enabled, the historical return profile is computed only from periods whose realised volatility over conditioning_volatility_window observations falls in the same of conditioning_quantile_bins bins as the current value, before the identical arithmetic of Sections 3.1 to 3.4. It ships off; turning it on is a measured change for a later release (DD-23).

4. Research, evidence, LLM, and E2B

4.1 Research chain and passages

```
Source-selection hierarchy
  -> exactly four neutral research categories
     (underlying_event, physical_flows, supply_response, market_reaction)
  -> grounded retrieval and safe URL resolution; publisher normalisation; duplicate handling
  -> strict-before-cutoff verification: maximum three attempts, verifier confidence at least 8/10,
     fail-closed per category
  -> verifier-cleaned summaries split into exact passages
  -> atomic claims with exact summary / passage / source linkage
  -> Python evidence policy
```

Passage extraction is a pure function of the cleaned summary text and the settings passage_min_chars and

passage_abbreviations (DD-13):

```
1  normalise   "\r\n" and "\r" become "\n"; runs of spaces and tabs collapse to one space; lines are stripped
2  segments    each non-empty line is a segment; a leading bullet marker ("-", "*", the bullet character,
               or digits followed by "." or ")") is removed
3  split       inside a segment, split after a terminator run [.!?]+ (plus any closing quotes or brackets)
               when it is followed by whitespace and then an uppercase letter, a digit, an opening quote,
               or an opening bracket. No split when the token ending in the terminator is in
               passage_abbreviations (case-sensitive). A period between two digits ($102.58) never splits
               because no whitespace follows it. An ellipsis is one terminator run
4  clean       strip each candidate; drop empties
5  merge       a candidate shorter than passage_min_chars is joined with one space to the following
               candidate; a short final candidate joins the preceding one. A summary shorter than the
               minimum yields exactly one passage
6  identify    passage_id = "SS.NN": SS = 1-based index of the accepted summary in packet order,
               NN = 1-based passage index, both zero-padded to two digits
```

- Passages are stored on the summary and included in the summary and packet fingerprints. A fixture of edge cases (decimals, abbreviations, bullets, ellipses, quotes, short fragments, single-sentence summaries) is tested.
- Passages never contain destination-page text. The Source Validator remains audit-only and its output never reaches the packet or the policy; the two v5.2 contract tests enforcing this are retained.
- Each material claim must cite at least one supporting summary, at least one supporting passage belonging to a cited summary, and at least one supporting source associated with that summary. Contradicting summaries, passages, and sources are recorded the same way.
- The Claim-Support Verifier audits each claim against its cited passages only. It stays audit-only and nonfatal.

4.2 Evidence policy

Evidence tier	Independent publishers	Minimum confidence	Maximum action
Tier 0	Requirements not met	N/A	Neutral only
Tier 1	At least 1	0.40	Small centre; small wider or narrower
Tier 2	At least 2	0.60	Moderate centre; moderately wider or narrower
Tier 3	At least 2	0.80	Large centre; substantially wider or narrower

- Tier 3 no longer requires three publishers, two publishers per claim, a primary source, physical-market evidence, or likely-new novelty (DD-12).
- Publishers are counted as the union of distinct normalised publishers across all valid claims cited by the horizon's action.
- Per-claim validity requires all six clauses: the claim is material; it links to at least one accepted summary; it lists at least one supporting source; at least one listed source is associated with a linked summary, present in the packet, and mechanically resolved; every listed source is associated with a linked summary; every cited passage belongs to a linked summary. The failing clauses of every rejected claim are recorded by name in the policy decision.
- A neutral proposal (no_change and unchanged) requires no evidence.
- Every non-neutral action requires at least one valid material claim, exact linkage, qualification under the relevant tier, and no material conflict. A material conflict forces Tier 0, no_change, and unchanged.
- Python caps proposals to the qualified tier and never upgrades them: tier N caps both the centre level and the uncertainty level at N; Tier 0 sets both levers neutral.
- Narrowing additionally requires a non-empty narrowing_uncertainty_resolved field regardless of tier.
- The tier evaluation (publishers, confidence, pass or fail per tier) is recorded in the policy decision.

4.3 Five independently configurable LLM roles

Role	Settings table	Default model	Temp.	Max tokens	Timeout
Main assessment	[llm.main]	gemini-3.1-flash-lite-preview	0.1	24,576	180 s
Research / search	[llm.search]	gemini-3.1-flash-lite-preview	0.1	4,096	60 s
Cutoff verifier	[llm.cutoff_verifier]	gemini-3.5-flash	0.0	4,096	60 s
Structured-output correction	[llm.structured_output_correction]	gemini-3.5-flash	0.0	8,192	60 s
Claim-Support Verifier	[llm.claim_support_verifier]	gemini-3.5-flash	0.0	4,096	60 s

Common generation parameter ([llm.common])	Locked value
Top-p	1.0

Candidate count	1
Frequency penalty	0
Presence penalty	0
Top-k	Unset
Stop sequences	Unset
LLM seed	Unset
Parallel tool calls	Disabled
LITE_MODEL alias	Not used; no import from aieng.forecasting.models

- Forwarding mechanism for the main role: the package constructs the ADK LiteLlm model object itself with model "openai/<main_model>", the proxy base URL and key from the environment, timeout, top_p, n, frequency_penalty, presence_penalty, parallel_tool_calls, and drop_params, and passes it as AgentConfig.model, which the shared factory accepts unchanged. Temperature and max output tokens go through AgentConfig. Shared main is not modified.
- The four auxiliary calls read their model, temperature, max tokens, timeout, and the common parameters from settings instead of literals.
- Role-specific settings must be forwarded to the actual invocation layer and recorded in the audit as the exact keyword arguments handed over. Storing settings without forwarding them is noncompliant. If the proxy drops a parameter, the audit records it as requested and dropped.
- Model availability is checked before any paid run (Section 10.2). Substituting another model for a configured one is prohibited everywhere in the package.
- The public predictor ID is agent_predictor_cfm_agent_v_6_0_<main_model>_continuous, computed from settings.
- Settings field names for models end in _model so the coach's discovery rule finds them.

4.4 Structured-output correction and fallback

```
Initial assessment
  +-- valid -> continue
  +-- invalid -> record validation errors
                 -> one correction-only resubmission (no tools; research and numerical suite are not rerun)
                       +-- valid -> continue
                       +-- invalid -> fully audited neutral fallback -> exact ensemble reproduction
```

- Audit both raw attempts, both sets of validation errors, the correction request, the retry result, and the final disposition.
- The neutral fallback uses center_action = no_change and uncertainty_action = unchanged for every horizon, confidence 0, no claims, novelty indeterminate.

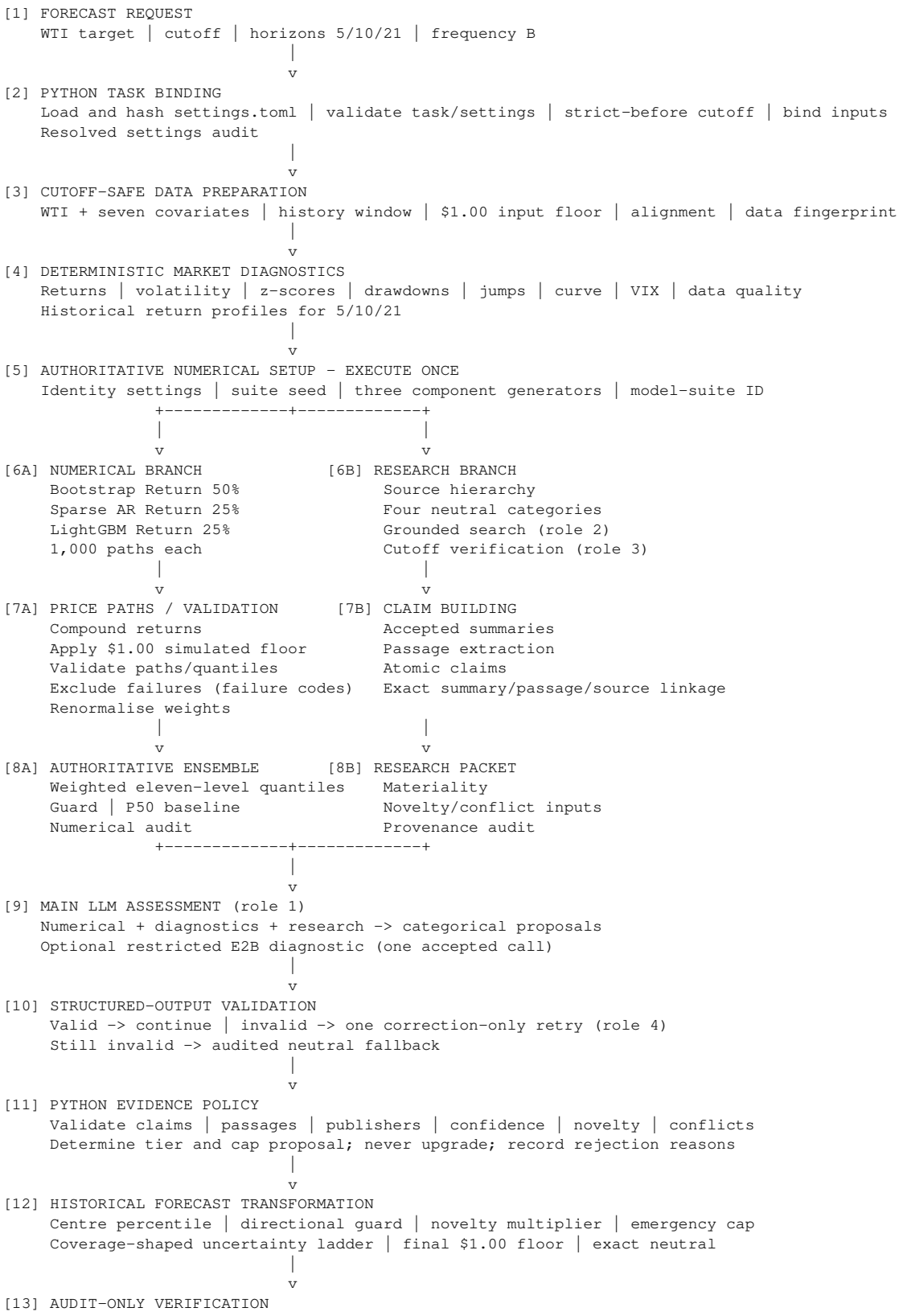
4.5 E2B controls

Control	**Locked default / requirement ([code_execution])**
Availability	Enabled but optional (code_execution_enabled, default true). The tool and the interpreter are constructed only when enabled, so the e2b dependency is not imported otherwise
Accepted executions	Maximum 1 (code_execution_max_accepted)
Rejected requests	Separately counted and audited; budget code_execution_max_rejected, default 3, after which further requests are refused and still recorded
Timeout	60 seconds per execution; sandbox lifetime 90 seconds
Maximum code length	10,000 characters
Maximum captured output	50,000 characters; truncation is flagged
Internet access	Off
Input payload	Python-defined: a preamble assigning DIAGNOSTICS, MODEL_SUITE_QUANTILES, and TASK_BINDING as JSON literals, followed by the LLM's script. Paths, research text, settings, and secrets are never included
Static screen	Purpose non-empty; length limit; the code parses; no import of or attribute access to any module in code_execution_prohibited_modules; no call to any name in code_execution_prohibited_calls; no use of any name in code_execution_prohibited_names (lists in Section 7)
Rejected request	Must not construct or call the interpreter; the LLM receives a JSON refusal naming every failed rule

Audit	Every request: index, purpose, code, code length, disposition, failed rules, launch status, duration, stdout, stderr, error, truncation flag; accepted and rejected lists kept separately

- Prohibit forecasting, task-binding recomputation, evidence decisions, transformations, final pricing, and filesystem, package, environment, skill, or runtime discovery.
- Prompt the LLM to plan the complete diagnostic upfront and use one self-contained script in the first accepted call. Prompt discipline supplements, but never replaces, Python-enforced budgets and guards.

5. v6.0 end-to-end flowchart



```
     Source Validator │ Claim-Support Verifier (role 5) │ nonbinding
                              │
                              v
[14] FINAL FORECAST AND COMPLETE AUDIT
     Eleven quantiles per horizon │ P50 point │ unified metadata record
```

6. Component groups, ownership, and package layout

Group	Components	Owner	Module
Configuration	settings.toml, loader, validation, identity field list, settings audit	Python	settings.toml, config.py
Public API	Agent configuration, predictor factory, public predictor	Python	agent.py, predictor.py
Task and data	Task binding, cutoff safety, WTI and covariates, window, alignment, floors, data fingerprint	Python	tools/market_data.py, history.py
Numerical suite	Three return models, path simulation, validation, ensemble, RNG, failure codes	Python	models/paths.py, bootstrap.py, sparse_ar.py, lightgbm_return.py, ensemble.py
Diagnostics	Returns, volatility, z-scores, drawdowns, jumps, current state, data quality, historical profiles	Python	diagnostics.py, history.py
Research	Source hierarchy, four queries, retrieval, cutoff verification, passages	LLM under Python controls	tools/research_pipeline.py, passages.py, source_registry.py, tools/url_safety.py, skills/
Claims	Atomic claims and exact summary/passage/ source linkage	LLM under Python validation	outputs.py, schemas.py
Assessment	Categorical centre, uncertainty, novelty, and conflict proposals	Main LLM role	prompts.py, skills/action-proposal
Correction	One schema-correction resubmission and neutral fallback	Correction LLM role	response_control.py
Evidence	Tier qualification, publisher counting, conflicts, caps, rejection reasons	Python	policy/evidence_policy.py
Transformation	Historical centre and coverage-shaped uncertainty arithmetic	Python	forecast_engine/transformation.py
LLM control	Five roles, parameter forwarding, effective-parameter audit, availability preflight	Python	llm_roles.py
E2B	Optional restricted diagnostics	LLM request; Python enforcement	tools/code_execution.py
Oversight	Source Validator and Claim-Support Verifier	Audit-only	source_validator.py, claim_support_verifier.py
Audit	Unified settings, execution, evidence, transformation, output record; canonical serialization	Python	predictor.py, fingerprints.py
Release	Protected-file baseline, data snapshot, model preflight, sensitivity report, manifest, archive, fresh-extraction retest	Python	release.py, PROTECTED_SCOPE.txt

Coach-facing contract, preserved without coach changes: config.AGENT_NAME, config.PACKAGE_ROOT, config.CfmV60Settings (pydantic, extra forbidden, frozen, bare construction returns the settings.toml values); agent.build_cfm_agent_config(data_service=, settings=, covariate_series_ids=, model=, search_model=) and agent.build_cfm_agent_predictor(config); forecast_engine.transformation.PythonForecastEngine(settings) with engine_id python_forecast_engine_v60; policy.evidence_policy.EvidencePolicy(settings) with policy_id graduated_evidence_policy_v60; outputs.CfmContextAssessmentOutput; schemas.ResearchPacket and schemas.ModelHorizonForecast; settings fields audit_enabled, policy_mode, code_execution_enabled; MANIFEST.sha256 at the package root. Registering v6.0 in cfm_coach (targets, streams, replay of the stored historical profile) is a separate, later task (DD-24).

Package documents: SPECIFICATION.pdf (this document), DECISIONS.md (Section 12), settings.toml (Section 7), README.md, ARCHITECTURE.md, CHANGELOG.md, INSTALL.md, BUILD_VERIFICATION.md, LIVE_VALIDATION.md, CALIBRATION.md, TRACEABILITY.md, CONTEXT.md (glossary), docs/adr/0001-coverage-shaped-uncertainty-ladder.md, PROTECTED_SCOPE.txt, PROTECTED_BASELINE.sha256, MANIFEST.sha256, five skills under skills/, tests under tests/.

7. Configuration file: settings.toml

7.1 Rule

- Every value that changes a forecast, an LLM call, an evidence decision, a research limit, an E2B budget, a diagnostic window, or a validation ceiling is a key in settings.toml at the package root. Python holds structural constants only: the eleven quantile levels (locked by the shared library), schema enumerations (action names, novelty classes, research categories), identifiers (engine, policy, control, and predictor ids), prompt text, and validation bounds (DD-16).
- TOML is used because it is read by the standard library (tomllib), carries comments, and types every value unambiguously. No new dependency is added.
- Key names equal CfmV60Settings field names. Tables ([numerical], [llm.main], ...) only group keys for reading; the loader flattens them and rejects a key that appears twice, a key that matches no field, or a field with no key. Mapping-valued fields (ensemble_weights, center_percentiles, novelty_multipliers, uncertainty_coverage) are written as their own table named after the field.
- A numeric literal other than 0, 1, -1, or 2 in a guarded module (models/, history.py, diagnostics.py, forecast_engine/, policy/, llm_roles.py, response_control.py, claim_support_verifier.py, passages.py, tools/code_execution.py, tools/research_pipeline.py) fails the literal-scan test unless the line carries the comment "# structural: <reason>".
- The comment tag [identity] marks values bound into the model-suite ID. The normative list is config.IDENTITY_FIELDS; a test parses the tags from the file and requires the two to agree.

7.2 Loading, precedence, and audit

- config.load_settings_file(path) returns the flattened mapping; config.py loads the packaged file once at import. CfmV60Settings(**overrides) applies explicit keyword overrides on top of the file values, so CfmV60Settings() reproduces the file exactly and the coach's bare construction keeps working. CfmV60Settings.from_file(path, **overrides) loads an alternate file for experiments.
- The environment variable CFM_V60_SETTINGS_FILE, when set, names the file used for the defaults; its use is recorded in the audit. Environment variables never override individual keys.
- Pydantic validation: bounds on every field; cross-field validators for weight keys and positivity, ladder strictly increasing with the anchor between the narrowing and widening rungs, centre percentiles strictly increasing with the down percentiles below 50 and the up percentiles above 50, exact key sets for the four mapping fields, final_price_floor at or below simulated_price_floor, and covariate_series_ids non-empty and distinct.
- config.resolved_settings_audit(settings) returns {settings_file, settings_file_sha256, values, overrides}, where overrides lists every field whose value differs from the packaged file. The manifest hashes settings.toml; the release gate fails if the packaged file's hash differs from the one recorded in BUILD_VERIFICATION.md.

7.3 The file

```
# CFM Agent v6.0 - settings.toml
# Single source of every tunable value. Key names equal CfmV60Settings field names; tables only
# group keys for reading. Python holds structural constants only (Section 7.1).
# [identity] marks a value bound into the model-suite ID: changing it changes every suite ID.
# Units: prices in USD per barrel; windows and lengths in WTI observations (trading days);
# percentiles and coverages in percent; timeouts in seconds.

[numerical]
model_num_paths = 1000                 # [identity] return paths per component; locked at 1000
simulation_length_business_days = 21   # [identity] steps per path; horizons 5/10/21 use the same paths
block_length_business_days = 5         # [identity] block size for bootstrap and residual sampling
history_window_business_days = 0       # [identity] 0 = entire cutoff-safe history; else trailing rows
input_price_floor = 1.0                # [identity] applied to prices before returns, diagnostics, fits
simulated_price_floor = 1.0            # [identity] applied to every simulated price before quantiles
final_price_floor = 1.0                # applied after transformation; must not exceed the simulated floor
require_all_components = false         # true = any component failure fails the whole suite
min_training_rows_margin = 50          # [identity] rows required beyond max lag + block before any fit
max_pre_execution_attempts = 2         # suite pre-execution attempts before the workflow fails
max_data_rows_per_series = 520         # rows shown to the LLM per series; models use the full window

[numerical.ensemble_weights]           # [identity] configured weights; renormalised over survivors
bootstrap = 0.50
sparse_ar = 0.25
lightgbm = 0.25

[numerical.sparse_ar]
return_lags = [1, 5, 10, 21, 63, 126, 252]   # [identity] lagged daily returns (AR and LightGBM features)
ar_ridge_penalty = 1e-5                      # [identity] ridge penalty on the seven coefficients
ar_fit_intercept = true                      # [identity] unpenalised intercept

[numerical.lightgbm]
```

```
lightgbm_n_estimators = 250            # [identity]
lightgbm_learning_rate = 0.03          # [identity]
lightgbm_num_leaves = 7                # [identity]
lightgbm_max_depth = 3                 # [identity]
covariate_forward_fill_limit = 5       # [identity] max consecutive gap rows forward-filled per covariate
covariate_series_ids = [               # [identity] shared-data-layer covariates, in feature order
  "brent_log_ret_1b_l1b", "natgas_log_ret_1b_l1b", "gasoline_log_ret_1b_l1b",
  "gold_log_ret_1b_l1b", "dollar_index_log_ret_1b_l1b", "oil_curve_contango_l1b",
  "vix_level_l1b",
]

[diagnostics]
diagnostic_return_windows = [1, 5, 10, 21, 63, 126, 252]
diagnostic_volatility_windows = [5, 10, 21, 63, 126, 252]   # volatility, z-score, and drawdown windows
diagnostic_jump_windows = [10, 21, 63, 126, 252]
trading_days_per_year = 252            # annualisation factor for realised volatility
curve_flat_threshold = 0.001           # abs(log USL/USO) below this labels the curve "flat"

[transformation]
emergency_center_cap_usd = 100.0             # absolute cap on the centre adjustment
emergency_center_cap_price_fraction = 1.0    # cap = min(cap_usd, fraction x latest WTI)
uncertainty_anchor_coverage = 80             # the ensemble P10-P90 band is equated to this coverage
min_historical_returns = 500                 # fewer eligible h-step returns forces a neutral transform
historical_conditioning = "none"             # "none" or "volatility_quintile" (ships off, DD-23)
conditioning_volatility_window = 21          # used only when conditioning is on
conditioning_quantile_bins = 5               # used only when conditioning is on

[transformation.center_percentiles]    # historical h-step return percentile selected per centre action
large_down = 10
moderate_down = 25
small_down = 40
small_up = 60
moderate_up = 75
large_up = 90

[transformation.novelty_multipliers]   # scales the centre adjustment before the emergency cap
likely_new_relative_to_model_data = 1.00
possibly_partly_reflected = 0.75
likely_reflected_in_model_data = 0.50
indeterminate = 0.50

[transformation.uncertainty_coverage]  # nominal historical coverage (percent) per uncertainty action
substantially_narrower = 65
moderately_narrower = 70
small_narrower = 75
unchanged = 80
small_wider = 85
moderately_wider = 90
substantially_wider = 95

[evidence]
tier_1_min_publishers = 1
tier_1_min_confidence = 0.40
tier_2_min_publishers = 2
tier_2_min_confidence = 0.60
tier_3_min_publishers = 2
tier_3_min_confidence = 0.80
policy_mode = "constrained_actions"

[research]                             # every other v5.2 research field is carried over here verbatim
research_max_results_per_query = 5
source_resolution_timeout_seconds = 8.0
source_resolution_max_redirects = 5
search_verifier_max_attempts = 3
search_verifier_confidence_threshold = 8     # of 10
passage_min_chars = 40
passage_abbreviations = [
  "U.S.", "U.K.", "U.N.", "E.U.", "Mr.", "Mrs.", "Ms.", "Dr.", "Inc.", "Ltd.", "Co.", "Corp.",
  "vs.", "e.g.", "i.e.", "St.", "No.", "Jan.", "Feb.", "Mar.", "Apr.", "Jun.", "Jul.", "Aug.",
  "Sep.", "Sept.", "Oct.", "Nov.", "Dec.", "bbl.", "approx.", "est.", "Gov.", "Sen.", "Rep.",
]

[llm.common]                           # forwarded with every role
llm_top_p = 1.0
llm_candidate_count = 1
llm_frequency_penalty = 0.0
llm_presence_penalty = 0.0
llm_parallel_tool_calls = false
```

```
[llm.main]                            # role 1: main assessment (ADK agent)
main_model = "gemini-3.1-flash-lite-preview"
main_temperature = 0.1
main_max_tokens = 24576
main_timeout_seconds = 180

[llm.search]                          # role 2: grounded research search
search_model = "gemini-3.1-flash-lite-preview"
search_temperature = 0.1
search_max_tokens = 4096
search_timeout_seconds = 60

[llm.cutoff_verifier]                 # role 3: strict-before-cutoff verification
cutoff_verifier_model = "gemini-3.5-flash"
cutoff_verifier_temperature = 0.0
cutoff_verifier_max_tokens = 4096
cutoff_verifier_timeout_seconds = 60

[llm.structured_output_correction]    # role 4: one correction-only resubmission
structured_output_correction_model = "gemini-3.5-flash"
structured_output_correction_temperature = 0.0
structured_output_correction_max_tokens = 8192
structured_output_correction_timeout_seconds = 60

[llm.claim_support_verifier]          # role 5: audit-only claim-support verification
claim_support_verifier_model = "gemini-3.5-flash"
claim_support_verifier_temperature = 0.0
claim_support_verifier_max_tokens = 4096
claim_support_verifier_timeout_seconds = 60

[code_execution]
code_execution_enabled = true
code_execution_max_accepted = 1
code_execution_max_rejected = 3
code_execution_timeout_seconds = 60
code_execution_sandbox_timeout_seconds = 90
code_execution_max_code_chars = 10000
code_execution_max_output_chars = 50000
code_execution_allow_internet = false
code_execution_prohibited_modules = [
  "os", "sys", "subprocess", "pathlib", "shutil", "importlib", "pkgutil", "site", "inspect",
  "socket", "http", "urllib", "requests", "ctypes", "multiprocessing", "signal", "glob",
  "tempfile", "builtins", "code", "runpy",
]
code_execution_prohibited_calls = [
  "open", "exec", "eval", "compile", "__import__", "globals", "locals", "vars", "breakpoint", "input",
]
code_execution_prohibited_names = [
  "environ", "getenv", "__builtins__", "__loader__", "__spec__", "__file__",
]

[audit]
audit_enabled = false

[validation]                          # read by release.py and the validation scripts only
live_spend_ceiling_usd = 3.0
sensitivity_excluded_dates = ["2020-04-20", "2020-04-21"]
```

7.4 Tests for the configuration

- test_settings_file.py: the file loads; CfmV60Settings() equals the file; every field has exactly one key and every key exactly one field; a duplicate key, an unknown key, and a missing key are rejected; every _model field is discoverable by suffix; the [identity] tags equal config.IDENTITY_FIELDS; the audit carries the file's SHA-256; an override is listed in overrides; the cross-field validators reject a final floor above the simulated floor, a non-increasing ladder, and a wrong key set.
- test_no_tunable_literals.py: parses every guarded module with ast and fails on any numeric literal outside {0, 1, -1, 2} whose line lacks a "# structural:" comment.

8. Audit record

Every prediction carries the v5.2 metadata keys (model_suite_id, numerical_suite_audit, research_execution_audit, code_execution_audit, llm_context_assessment, structured_output_control_audit, active_research_packet, cutoff_verification_audit, source_validation_audit, claim_support_audit, claim_support_findings, policy_decision, forecast_transformation, unadjusted_ensemble, market_diagnostics, task_binding, audit_switch, audit_only_controls,

active_evidence_declaration, audit_findings_changed_forecast = False, llm_authored_final_numbers = False) plus: resolved_settings (settings_file, settings_file_sha256, values, overrides), floors (settings and activation counts), history_window, data_fingerprint, data_snapshot (path and hash when a frozen snapshot was used, else null), component_seeds, component_dispositions (status, failure code, verbatim reason), ensemble_guard_applied, historical_transformation_inputs (the per-horizon profile), llm_effective_parameters (per role, as forwarded, with anything the proxy dropped), llm_preflight (per model, available or not, when run), code_execution_requests (accepted and rejected). The per-horizon transformation record adds the selected centre percentile and value, the directional-guard flag, the novelty multiplier, the emergency cap, the coverage target, anchor band, target band, uncertainty multiplier, and any neutral-forcing reason.

9. Development plan: direct v5.2 to v6.0, one release

The direct path is selected (DD-02). There are no intermediate releases and no intermediate approvals. Each step ends in one git commit; shared main, v5.0, v5.2, and cfm_coach are never modified.

Step	Technical work	Exit criterion
0	Commit cfm_agent_v_5_2 and IMPROVEMENTS.md, and nothing else, as the baseline commit	v5.2 is tracked; its manifest verifies
1	Write PROTECTED_SCOPE.txt and snapshot protected files into PROTECTED_BASELINE.sha256; copy this document in as SPECIFICATION.pdf; write DECISIONS.md, the TRACEABILITY.md skeleton, and CONTEXT.md; scaffold the package from a copy of v5.2 with all identifiers renamed; write settings.toml, config.py, and the schemas	Every requirement in this document has a traceability row; the package imports; the configuration tests pass
2	history.py, canonical serialization, the three models, paths, ensemble, failure codes, RNG, suite tool, numerical audit	Focused numerical, failure, renormalisation, and determinism tests pass
3	Expanded diagnostics and data quality	Diagnostic and cutoff tests pass
4	Transformation engine, historical profile, ADR	Percentile, guard, novelty, cap, ladder, floor, neutral, and replay tests pass
5	Evidence tiers, passage extraction in the research pipeline, rejection reasons	Boundary, conflict, narrowing, cap, and passage tests pass
6	Five LLM roles, parameter forwarding, availability preflight	Effective invocation parameters verified by test
7	E2B hardening, audit expansion, prompts and skills	Budget, screen, and no-launch-on-rejection tests pass
8	Public predictor integration; offline end-to-end test; documents; literal-scan test green	Offline end-to-end test passes
9	Static checks, full package suite, inherited v5.2 suite, data snapshot, cross-process determinism on the snapshot, production-data check, model preflight, live validation under the ceiling with cost accounting, sensitivity report; fill BUILD_VERIFICATION.md, LIVE_VALIDATION.md, CALIBRATION.md	All gates of Section 10 pass except packaging
10	Verify protected files, clean caches, write the manifest, package the archive, freshly extract and rerun	Clean-extraction validation passes

- PROTECTED_SCOPE.txt is a gitignore-style list of include and exclude patterns, read by release.py and hashed in the manifest, so every developer hashes the same files. Its content: include aieng-forecasting/**, implementations/energy_oil_forecasting/*.py, implementations/energy_oil_forecasting/specs/**, implementations/pyproject.toml, pyproject.toml, uv.lock, Makefile, .pre-commit-config.yaml, implementations/energy_oil_forecasting/cfm_agent_v_5_0/**, implementations/energy_oil_forecasting/cfm_agent_v_5_2/**, implementations/energy_oil_forecasting/cfm_coach/**/*.py, implementations/energy_oil_forecasting/cfm_coach/**/*.md; exclude **/.venv/**, **/__pycache__/**, **/*.pyc, **/.DS_Store, and the cfm_coach run, calibration, history, proposal, and challenger data directories, which change daily.
- Release tooling lives in the package as release.py with commands snapshot-protected, verify-protected, snapshot-data, preflight-models, sensitivity, write-manifest, verify-manifest, clean-caches, package, and verify-archive. The manifest format and exclusions match cfm_coach/review/challenger.py. Archives, data snapshots, and sensitivity outputs are written under implementations/energy_oil_forecasting/outputs/cfm_agent_v_6_0/ and are not committed.
- Data snapshot: snapshot-data copies the yfinance cache directory to outputs/cfm_agent_v_6_0/data_snapshot_<UTC>/ and writes DATA_SNAPSHOT.sha256 beside it. Every validation run in step 9 passes that directory as cache_dir to the shared data-layer builder, so the coach's daily job cannot change the data between runs. Market data is never committed (DD-20).

- Test conventions follow v5.2: hand-written stubs and monkeypatch, no mock library; every package-owned v5.2 test file (conftest.py, test_audit_controls.py, test_contract.py, test_corrective_controls.py, test_policy_and_engine.py, test_research_pipeline.py, test_v51_improvements.py, test_v51_regressions.py) is preserved and adapted, and TRACEABILITY.md maps each v5.2 test to its v6.0 counterpart (DD-21); a synthetic deterministic price and covariate service is used offline.

10. Mandatory gates

Gate	Requirement
Isolation	New sibling package; shared main, v5.0, v5.2, and cfm_coach unchanged, verified against PROTECTED_SCOPE.txt, PROTECTED_BASELINE.sha256, and the v5.0 and v5.2 manifests
Traceability	Every normative requirement of this document mapped to code and executable evidence in TRACEABILITY.md, with deviations from the original planning documents listed (Section 12)
Configuration	settings.toml and CfmV60Settings agree field for field; the literal-scan test passes; the packaged file's hash is recorded
Static quality	Compilation, public imports, Ruff format, Ruff lint
Testing	Focused, edge-case, failure-path, contract, and inherited v5.2 regression tests; the v5.2 package suite still passes unchanged
Numerical validation	Paths, horizons, parameters, floors, failures, ensemble arithmetic; on the frozen data snapshot, two fresh processes reproduce identical data fingerprint, seeds, quantiles, and suite ID; identity setting changes alter the ID, LLM settings do not; a differing data fingerprint is reported as a data change, never as a determinism failure
Offline integration	Complete public-predictor end-to-end execution with stubbed LLM and search, including the correction and neutral-fallback paths
Production data	Cutoff-safe WTI and seven-covariate validation on the data snapshot; the suite runs on real data in under one minute
Model availability	preflight-models confirms every configured role model through the proxy before any paid run; a missing model stops the build (Section 10.2)
Live integration	Gemini via the proxy, grounded search, cutoff verification, one E2B execution, Langfuse trace; two identical runs in fresh processes agree numerically; the correction path exercised live; effective parameters recorded. Total spend capped at live_spend_ceiling_usd under the accounting of Section 10.1
Sensitivity	release.py sensitivity run at the release cutoff and its table recorded in CALIBRATION.md
Protection	Authoritative baseline and shared files verified unchanged
Packaging	Cache cleanup, manifest, archive hash, fresh extraction, full retest
Approval	No release before all mandatory gates pass and BUILD_VERIFICATION.md and LIVE_VALIDATION.md record the results

10.1 Live-validation cost accounting

- Counted usage: every LLM generation in the run's Langfuse trace, by model, input tokens, and output tokens. E2B sandbox time is recorded but not priced.
- Prices: the per-million-token input and output rates for the two configured models, taken from the proxy's published price list, or from the provider's public list price for the same model names when the proxy publishes none. The rates, their source, and the retrieval date are copied into LIVE_VALIDATION.md.
- Planned runs: preflight (unpriced), run 1 (full), run 2 (identical, weight 1.0), correction path (weight 0.2). Projection after run 1: projected total = actual(run 1) × 2.2.
- Stop condition: if actual spend plus the projection for the remaining runs exceeds live_spend_ceiling_usd, no further paid call is made and the owner is asked. Actual spend per run is recorded (DD-22).

10.2 Model availability preflight

- release.py preflight-models lists the models exposed by the proxy (GET <base URL>/models) and checks every distinct configured role model. If the listing endpoint is unavailable, it sends one completion with max_tokens = 1 per distinct model instead.
- Any configured model that is not available stops the build; substitution is prohibited. Results are recorded in LIVE_VALIDATION.md and in the llm_preflight audit key of the validation runs.

```
uv run ruff format --check implementations/energy_oil_forecasting/cfm_agent_v_6_0
uv run ruff check implementations/energy_oil_forecasting/cfm_agent_v_6_0
uv run python -m compileall -q implementations/energy_oil_forecasting/cfm_agent_v_6_0
uv run pytest -q implementations/energy_oil_forecasting/cfm_agent_v_6_0/tests
```

```
uv run pytest -q implementations/energy_oil_forecasting/cfm_agent_v_5_2/tests
uv run python -m energy_oil_forecasting.cfm_agent_v_6_0.release verify-protected
uv run python -m energy_oil_forecasting.cfm_agent_v_6_0.release snapshot-data
uv run python -m energy_oil_forecasting.cfm_agent_v_6_0.release preflight-models
uv run python -m energy_oil_forecasting.cfm_agent_v_6_0.release sensitivity
uv run python -m energy_oil_forecasting.cfm_agent_v_6_0.release verify-manifest
uv run python -m energy_oil_forecasting.cfm_agent_v_6_0.release package
uv run python -m energy_oil_forecasting.cfm_agent_v_6_0.release verify-archive
```

11. Out of scope for this release

- Registering v6.0 in cfm_coach (targets, streams, replay support), scheduling any run, or changing the coach in any way.
- Any backtest or skill measurement beyond the gates above.
- Turning regime conditioning on.
- Staged v5.8 or v5.9 releases.
- Any change to the shared data layer, the aieng-forecasting library, v5.0, or v5.2, including the cutoff convention.

12. Design-decision register

Each entry records what was decided, what was considered instead, and why. Entries marked "deviates" change something the original v6.0 planning documents asked for. The register is mirrored in DECISIONS.md; a later change to any decision is appended there as a new dated entry that supersedes the old one, never by editing the old entry.

ID	Decision	Considered instead	Reason and consequence
DD-01	v6.0 is a new isolated sibling package; v5.2 and IMPROVEMENTS.md are committed first as the git baseline	Modify v5.2 in place	The coach runs v5.2 daily; byte-for-byte protection needs a committed baseline. Consequence: two packages coexist
DD-02	One release, directly from v5.2 (deviates)	Staged v5.8, v5.9, v6.0	Every exit criterion is testable in one package; staging triples packaging work and changes no requirement
DD-03	Three return-path models replace the Darts price-level models	Keep Darts models and fix calibration	Measured: the v5.2 base sat about $2.60 below the last price at h=21 because price-level LightGBM cannot extrapolate, and the suite was not deterministic. Return paths are extrapolation-free and fully seeded
DD-04	Weights 0.50/0.25/0.25; 1,000 paths; five-step blocks; lags 1 to 252	Equal weights; longer blocks	Values from the planning documents; the bootstrap is the least model-dependent component and carries the most weight. All are settings
DD-05	Eleven quantile levels are produced (deviates: the planning documents listed five)	Five levels	The shared library, the coach, and v5.2 output use eleven levels; the five are a subset. The transformation arithmetic uses only P10 and P50
DD-06	$1.00 input, simulated, and final floors; no return clipping; the April 2020 observations stay in every fit; a sensitivity report is produced at release (deviates: planning documents said $0.01)	$0.01 floor; excluding or winsorising April 2020	Owner decision to keep the literal rule. Consequence, measured: AR residual sd 12.5% versus 2.6% and intercept +0.224% versus +0.055% per day; 2 of 1,000 bootstrap paths above twice the price. Recorded, informational, not corrected
DD-07	Full cutoff-safe history by default with one optional window setting; the LLM snapshot stays 520 rows	520-row window for everything	The 520 limit in v5.2 only clamps the LLM snapshot; models always trained on full history. Full-history bands cover 78 to 86% of the last three years of moves against an 80% target
DD-08	Sampled returns keep the history's drift	Demean the returns	Owner decision to keep the literal rule. Consequence: the ensemble P50 sits about $1.50 to $2.00 above the last price at h=21
DD-09	Centre adjustment = P50 × historical percentile of h-step returns, with a directional guard	Width-fraction steps (v5.2); no guard	Derived from data rather than hand-set. The guard is needed: a trailing-window P40 was positive in 22% of 520-day windows, which would have moved a small-down action up

DD-10	Coverage-shaped uncertainty ladder anchored at the historical 80% band, applied as a shape ratio to the ensemble's own distances (deviates)	Literal dollar targets from absolute-deviation percentiles P10 to P90	Measured: the literal targets made small and moderate widening no-ops (factors 0.60 and 0.83) and cut widths by 62 to 90% on narrowing. The ladder keeps the ensemble's shape and moves coverage by a known amount; ADR 0001
DD-11	Novelty multipliers 1.0/0.75/0.5/0.5 apply to all four classes; no eligibility gate; no tier cap; novelty never qualifies evidence (deviates)	Keep the v5.2 gate that only two classes may act	The multiplier table is meaningless under the gate. Owner decision not to cap by tier
DD-12	Tier 3 = two publishers and 0.80 confidence; shared cap ladder; conflict forces Tier 0; narrowing needs a resolved uncertainty	Keep v5.2's Tier 3 (three publishers, primary source, physical claim, likely-new novelty)	v5.2's Tier 3 was never reached in 40 live runs; the cap ladder is shown in code and kept
DD-13	Passages are sentence-level slices of verifier-cleaned summaries, produced by a defined deterministic algorithm	Destination-page excerpts; LLM-chosen quotes	Destination text must stay outside the active path (v5.2 contract tests). A defined algorithm makes passage ids stable across runs and machines
DD-14	Five LLM roles, each with model, temperature, max tokens, and timeout; temperatures 0.1/0.1/0.0/0.0/0.0; seed unset; forwarding via a package-built LiteLlm passed as AgentConfig.model	One shared model alias; modifying the shared factory	Verification and correction roles must be exact, so 0.0; the two generative roles use 0.1 to avoid degenerate repetition in long structured output. The shared factory accepts a model object, so shared main stays untouched. Seeds are unset because the proxy does not honour them
DD-15	E2B: Python-defined payload, static prohibited-operation screen, one accepted and three rejected requests, 60 s, 10k chars in, 50k out, internet off	Unlimited LLM-authored execution (v5.2); removing E2B	Owner decision to keep the LLM as author and Python as enforcer. Rejected requests never launch a sandbox
DD-16	Every tunable value lives in settings.toml; key = field name; Python keeps structural constants only; a literal-scan test enforces it	Python defaults in config.py (v5.2); YAML; pydantic-settings with environment variables	Owner requirement: one readable place for model settings and calibration inputs. TOML needs no dependency and carries comments. The file is hashed into the audit and the manifest, so a run is tied to an exact configuration
DD-17	One canonical JSON encoder for every identity payload; failure codes, not error text, enter the suite identity	v5.2's encoder with str() fallback	Readiness-assessment finding: str() fallbacks and error messages can differ across processes. Consequence: identity payloads with unsupported types fail loudly
DD-18	Fixed order of operations for quantiles, the guard, P50, and the floors; the guard is defensive and may not change anything; final floor at or below the simulated floor	Leave the order to the implementation	Readiness-assessment finding. The floor ordering makes exact neutral and the final floor provably compatible
DD-19	A step is one WTI observation; forecast dates use pandas business days	An exchange calendar	The shared library and its cutoff enforcer already use pandas business days; an exchange calendar would change the shared data layer, which is out of scope. The holiday mismatch is at most one day per horizon
DD-20	Release validation runs on a frozen, hashed copy of the yfinance cache; the determinism gate is conditional on an equal data fingerprint	Run on the live cache; commit market data	The coach's daily job shares the cache directory and could rewrite it between runs; market data does not belong in git
DD-21	All seven v5.2 test files are preserved and mapped; the protected scope is a machine-readable file; release tooling lives in the package	Five named files; scope in prose; a repository-level tool	Two v5.2 files (v51 improvements and regressions) test behaviour v6.0 keeps. No packaging tooling exists in the repository
DD-22	Live validation under a $3 ceiling with defined cost accounting and a model preflight	No live validation; unbounded spend	Owner decision on the ceiling; the accounting rule makes the stop condition mechanical
DD-23	Volatility-regime conditioning of the historical profile ships as a setting, off	On by default; absent	Measured two-fold width variation across volatility regimes, but turning it on is a change that needs scored evidence first
DD-24	No coach registration, streams, replay support, or scheduled runs in this release; coach-facing module layout preserved	Register v6.0 in the coach now	Owner decision; the preserved layout makes later registration a coach-side task only

13. Glossary

Term	Meaning
Return path	One simulated sequence of 21 daily simple returns produced by a component
Step	One WTI observation (a trading day); horizons count steps, forecast dates count pandas business days
Component	One of the three return models; each yields 1,000 return paths and eleven quantiles per horizon
Eligible return	A horizon-matched cumulative return computed from cutoff-safe, floored prices inside the history window
Historical return profile	Per horizon: eligible count, window bounds, median, centre percentiles, and absolute-deviation bands
Coverage rung	An uncertainty action expressed as the nominal share of historical moves its band contains
Anchor band	The historical 80% band, to which the ensemble's own P10-to-P50 half-width is equated
Directional guard	The rule that a centre action can only move the centre in its own direction
Evidence tier	The Python-determined qualification level (0 to 3) that caps both levers
Passage	A sentence-level slice of a verifier-cleaned summary, with identifier SS.NN
Accepted / rejected execution	An E2B request that passed or failed the Python screen; only accepted requests launch a sandbox
Neutral transformation	A literal copy of the authoritative ensemble, produced when both actions are neutral or when history is insufficient
Failure code	The fixed-vocabulary reason a component was excluded; part of the suite identity, unlike the verbatim message
Identity setting	A settings.toml key tagged [identity]; its value is bound into the model-suite ID
Settings file	settings.toml, the single home of every tunable value; hashed into the audit and the manifest
Data snapshot	A frozen, hashed copy of the yfinance cache used for release validation runs
Protected file	A file outside the package, selected by PROTECTED_SCOPE.txt, whose hash must be unchanged at release
Model-suite ID	The SHA-256 identity of the numerical suite: task, identity settings, data fingerprint, seeds, and results
Data fingerprint	The SHA-256 of the canonical serialization of the floored, windowed, cutoff-safe observations the suite received